LAB SHEET 2

Question 1

Create a script.js file. In it, define a JavaScript array of product objects (id, name, price, image) representing your store's catalog — at least 8 products.

Answer:

```
const products = [
    { id: 1, name: "T-Shirt", price: 799, image: "tshirt.jpg" },
    { id: 2, name: "Sneakers", price: 1999, image: "shoes.jpg" },
    { id: 3, name: "Backpack", price: 1299, image: "bag.jpg" },
    { id: 4, name: "Watch", price: 2499, image: "watch.jpg" },
    { id: 5, name: "Headphones", price: 1599, image: "headphones.jpg" },
    { id: 6, name: "Cap", price: 499, image: "cap.jpg" },
    { id: 7, name: "Hoodie", price: 1499, image: "hoodie.jpg" },
    { id: 8, name: "Wallet", price: 699, image: "wallet.jpg" }
];
```

Question 2

On products.html, use JavaScript to dynamically render the product cards from your array into the page (instead of hardcoded HTML blocks) using DOM methods like createElement / innerHTML with template literals.

Answer:

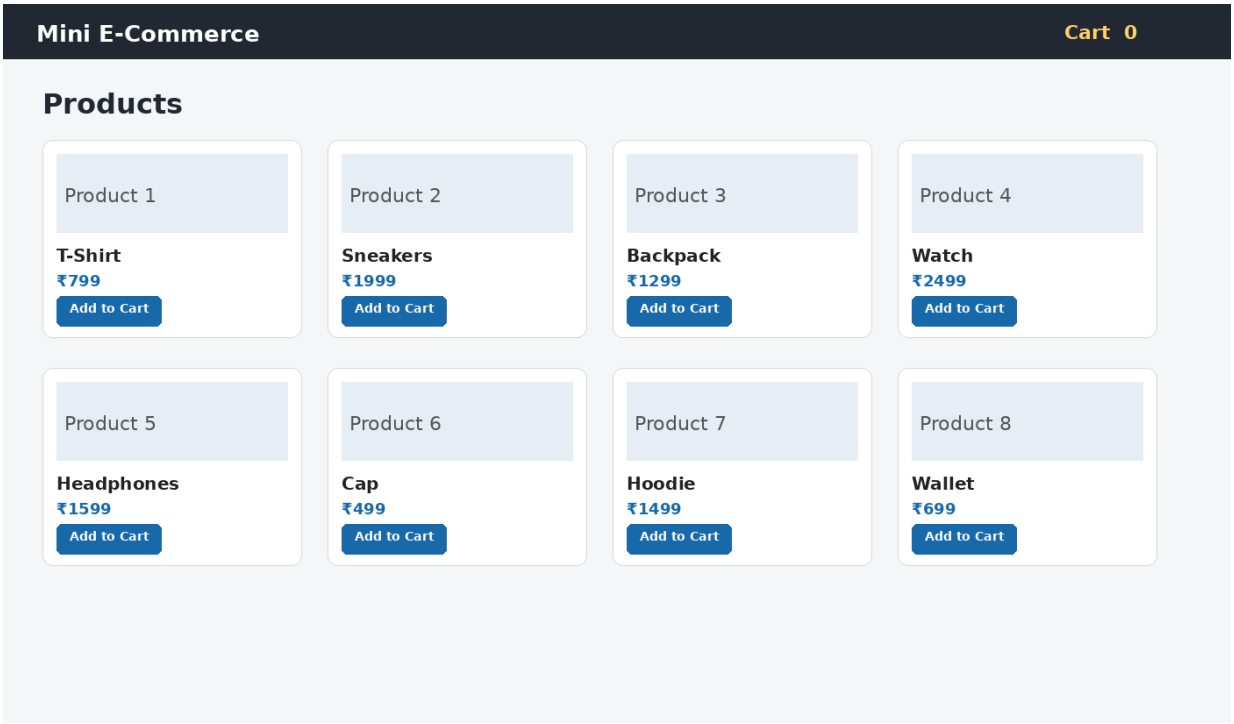
```
const productContainer = document.querySelector("#products");

products.forEach(product => {
    const card = document.createElement("div");
    card.className = "product-card";

    card.innerHTML = `
        <img src="${product.image}" alt="${product.name}">
        <h3>${product.name}</h3>
        <p>■${product.price}</p>
        <button>Add to Cart</button>
    `;

    productContainer.appendChild(card);
});
```

Output:



Question 3

Implement an 'Add to Cart' click handler on each product card that adds the product to a cart array (store it in a JS variable, and also persist it using localStorage so it survives a page refresh).

Answer:

```
let cart = JSON.parse(localStorage.getItem("cart")) || [];

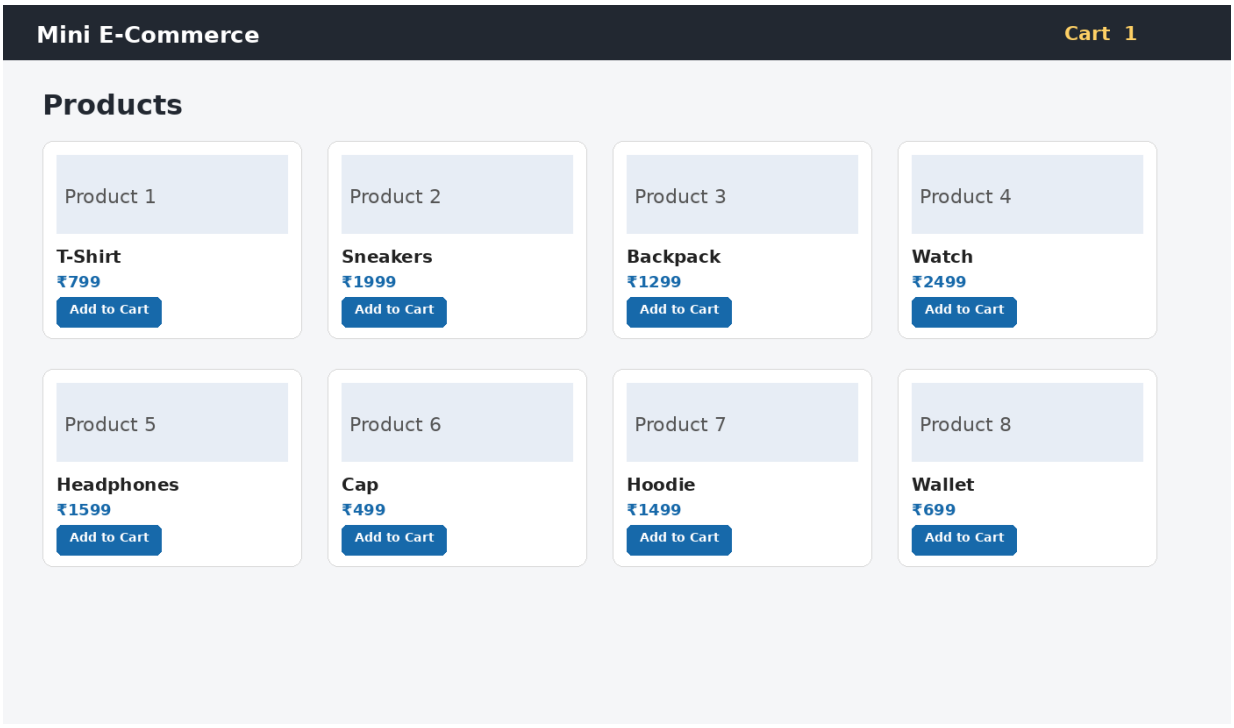
function addToCart(product) {
    const item = cart.find(p => p.id === product.id);

    if (item) {
        item.quantity++;
    } else {
        cart.push({ ...product, quantity: 1 });
    }

    localStorage.setItem("cart", JSON.stringify(cart));
}

document.querySelectorAll(".add-cart").forEach(button => {
    button.addEventListener("click", () => {
        const id = Number(button.dataset.id);
        const product = products.find(p => p.id === id);
        addToCart(product);
    });
});
```

Output:



Question 4

On cart.html, read the cart from localStorage and dynamically render each item with a working quantity input and a 'Remove' button.

Answer:

```
let cart = JSON.parse(localStorage.getItem("cart")) || [];
const cartContainer = document.querySelector("#cart");

cart.forEach(item => {
    const div = document.createElement("div");

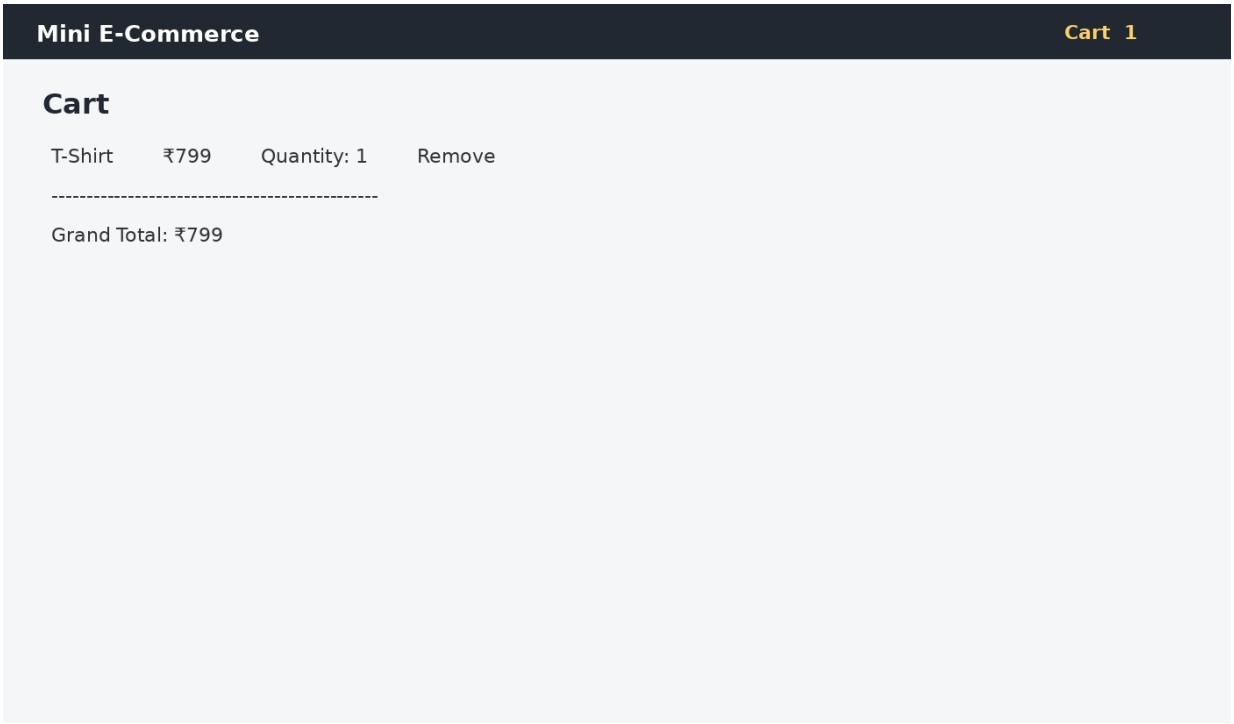
    div.innerHTML = `
        <h3>${item.name}</h3>
        <input type="number" min="1" value="${item.quantity}">
        <button class="remove">Remove</button>
    `;

    div.querySelector("input").addEventListener("change", e => {
        item.quantity = Number(e.target.value);
        localStorage.setItem("cart", JSON.stringify(cart));
        calculateTotal();
    });

    div.querySelector(".remove").addEventListener("click", () => {
        cart = cart.filter(p => p.id !== item.id);
        localStorage.setItem("cart", JSON.stringify(cart));
        div.remove();
        calculateTotal();
    });

    cartContainer.appendChild(div);
});
```

Output:



Question 5

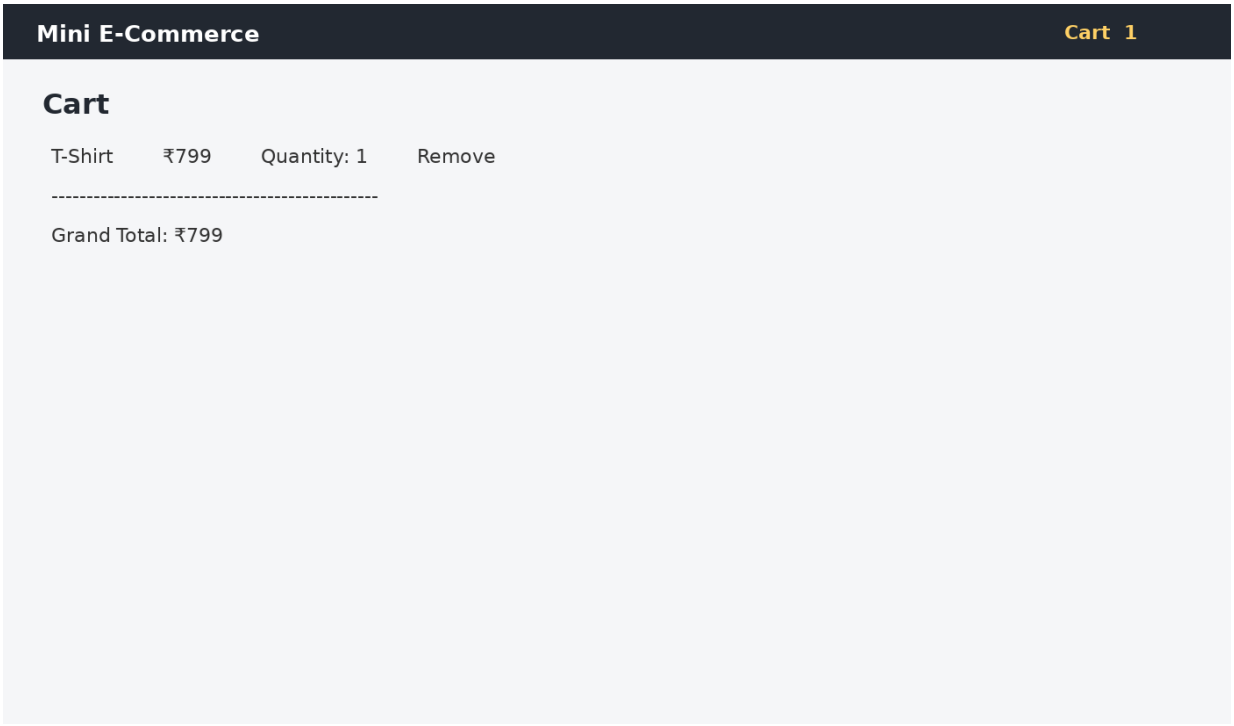
Write a function calculateTotal() that recalculates and displays the cart's grand total whenever quantity changes or an item is removed.

Answer:

```
function calculateTotal() {
    const total = cart.reduce(
        (sum, item) => sum + item.price * item.quantity,
        0
    );

    document.querySelector("#grand-total").textContent =
        "Grand Total: ■" + total;
}
```

Output:



Question 6

On checkout.html, add form validation using JavaScript: Name and Address must not be empty, Pincode must be exactly 6 digits, Phone must be exactly 10 digits. Show inline error messages instead of using alert().

Answer:

```
const form = document.querySelector("#checkoutForm");

form.addEventListener("submit", function (e) {
    e.preventDefault();

    let valid = true;

    document.querySelector("#nameError").textContent = "";
    document.querySelector("#addressError").textContent = "";
    document.querySelector("#pincodeError").textContent = "";
    document.querySelector("#phoneError").textContent = "";

    if (name.value.trim() === "") {
        nameError.textContent = "Name is required";
        valid = false;
    }

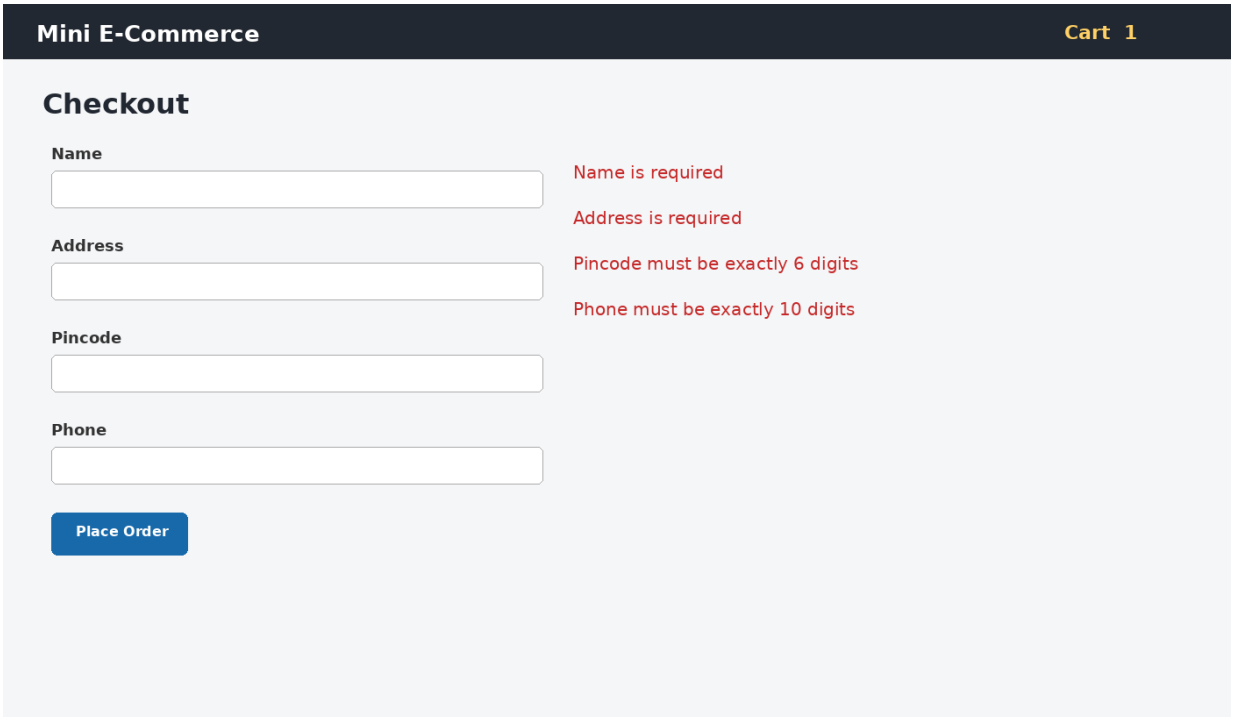
    if (address.value.trim() === "") {
        addressError.textContent = "Address is required";
        valid = false;
    }

    if (!/^\d{6}$/.test(pincode.value)) {
        pincodeError.textContent = "Pincode must be exactly 6 digits";
        valid = false;
    }

    if (!/^\d{10}$/.test(phone.value)) {
        phoneError.textContent = "Phone must be exactly 10 digits";
        valid = false;
    }

    if (valid) {
        document.querySelector("#message").textContent = "Order placed!";
    }
});
```

Output:



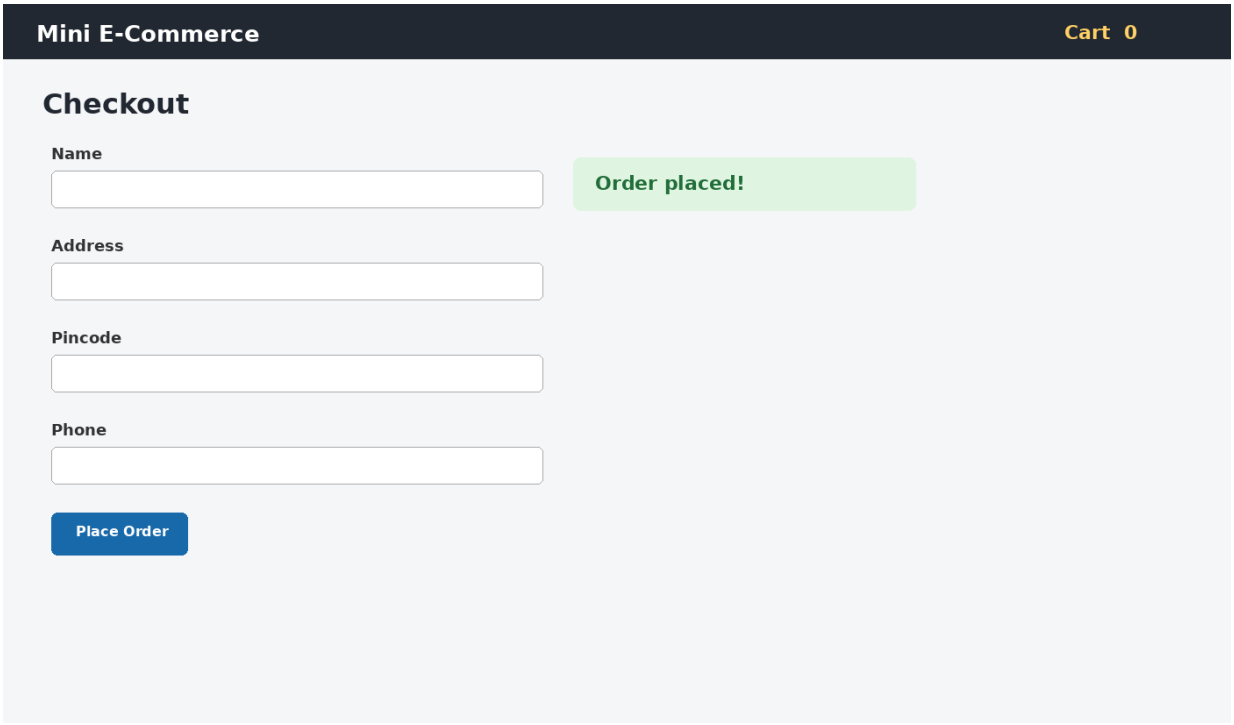
Question 7

On successful validation, show a simple confirmation message on the page (e.g., 'Order placed!') and clear the cart from localStorage.

Answer:

```
if (valid) {
    document.querySelector("#message").textContent = "Order placed!";
    localStorage.removeItem("cart");
}
```

Output:



Question 8

Add a small script on every page to update a 'cart item count' badge next to the Cart link in the navbar, read from localStorage.

Answer:

```
function updateCartCount() {
    const cart = JSON.parse(localStorage.getItem("cart")) || [];

    const count = cart.reduce(
        (total, item) => total + item.quantity,
        0
    );

    document.querySelector("#cart-count").textContent = count;
}

updateCartCount();
```

Output:

